

Agenda:

1. What is backtracking

2. Intro to Subset:

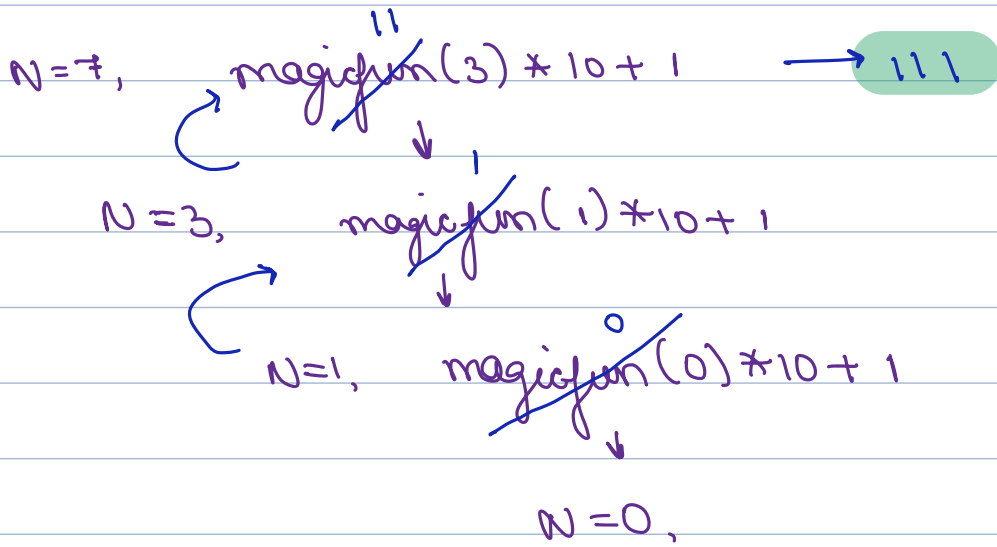
3. Return all subset array in an array.

4. Returns all permutation in an array.

Quiz 1:

```
int magicfun(int N){  
    1. if(N==0)  
        return 0;  
    2. return magicfun(N/2)*10 + (N%2);  
}
```

$O(\log N)$



Quiz 2:

T.C.

$N \rightarrow N/2 \rightarrow N/4 \rightarrow N/8 \dots 0$

$O(\log_2 N)$

[First value printed]

Quiz 3: void fun(char s[], int a) {

SCROLL, 0

print(s);

char temp;

if (a < s.length/2) {

temp = s[a];

s[a] = s[s.length - a - 1];

s[s.length - a - 1] = temp;

fun(s, a+1);

}

}

Output

SCROLL

LCROLS

LLROCS

LLORCS

0 1 2 3 4 5
SCROLL, 0.

$0 < 6/2 \rightarrow \text{True}$

swap(0, 5)

↓

0 1 2 3 4 5
(LCROLS, 1)

$1 < 6/2 \rightarrow \text{True}$

swap(1, 4)

↓

0 1 2 3 4 5
(LLROCS, 2)

$2 < 6/2$

swap(2, 3)

↓

(LLORCS, 3)

$3 < 6/2 \rightarrow \text{False}$

Quiz 4: TC. $(N/2) \rightarrow O(N)$

()

Ques: Print all valid parenthesis of length $2N$ for a given value of N .

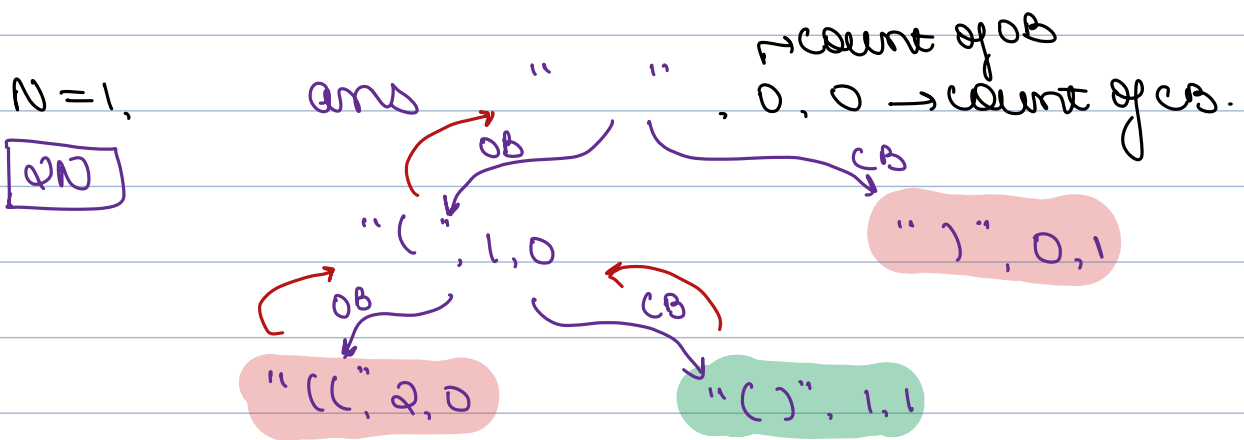
↳ equal no. of opening & closing brackets

$N=1$ (),) (

$N=2$ ()(), (()),))((,))(,)))

$N=3$ ()()(), ((())), ((())), (())(), (())(), (())()

Idea: Start generating string starting from 0 till $len=2N$



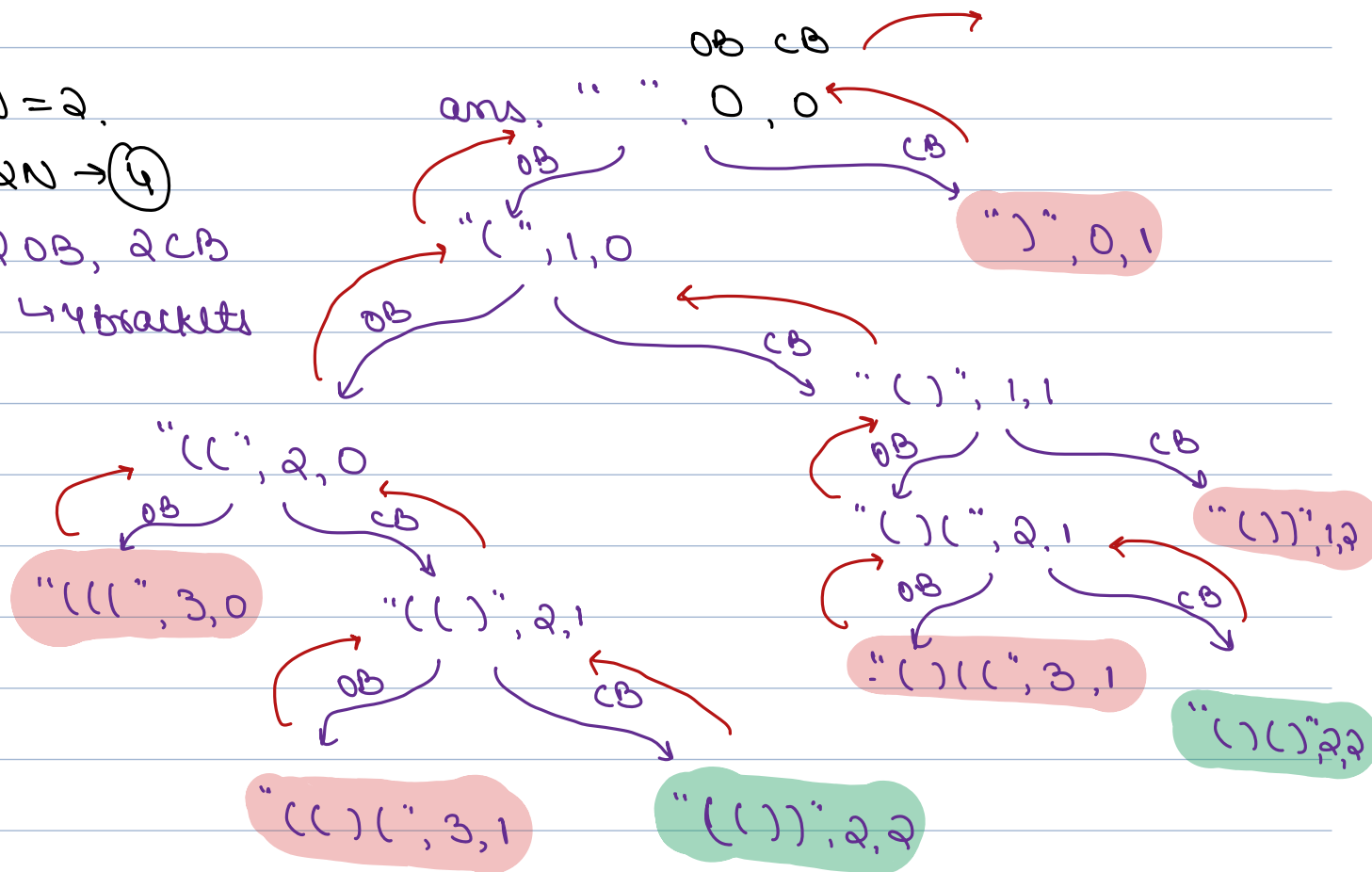
Invalid

1. opening > N
2. closing > opening

$$N=2.$$
$$\mathbb{Q} \subset \mathbb{R} \rightarrow \mathbb{Q}$$

2 OB, 2 CB

↳ 4 brackets



code:

```
void solve(int n, string ans, int open, int close){
    if (ans.length() == 2 * n){
        solve(ans);
        return;
    }
    if (open < n){
        solve(n, ans + "(", open + 1, close);
    }
    if (close < open){
        solve(n, ans + ")", open, close + 1);
    }
}
```

3

$$N = 2,$$
$$(2, \dots, 0, 0)$$

OB

$$(2, "(", 1, 0)$$

02

CB

$$(2, (1, 2, 0))$$
$$(\varnothing, "()", 1, 1)$$

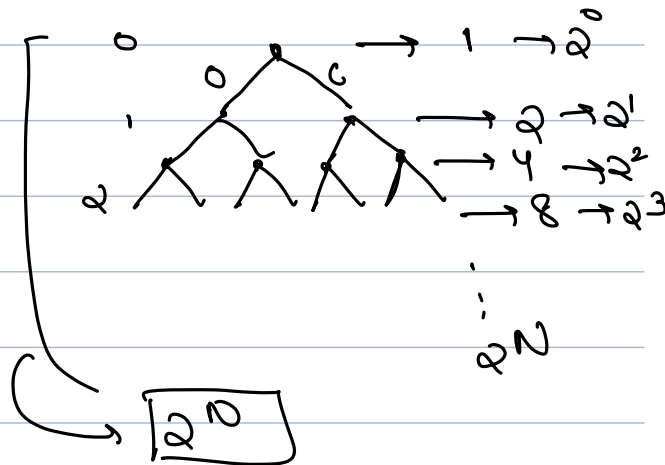
00

$$(2, "((", 2, 1)$$
$$(2, "()()", 2, 1)$$
$$(2, "(())", 2, 2)$$
$$(2, "()", 2, 2)$$

TODO : $N=3$

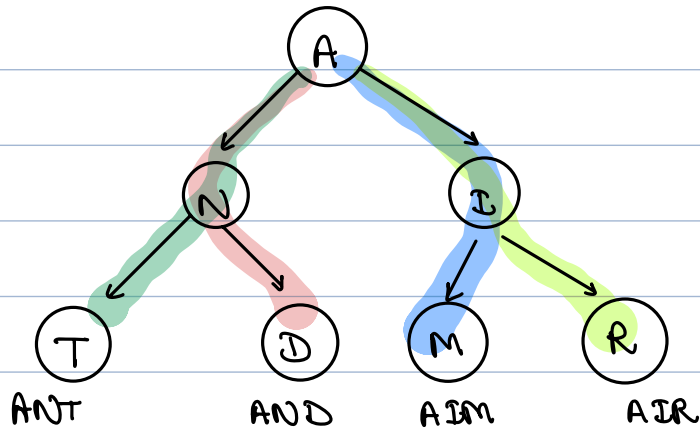
TC: $O(2^N)$

sc: $O(N)$



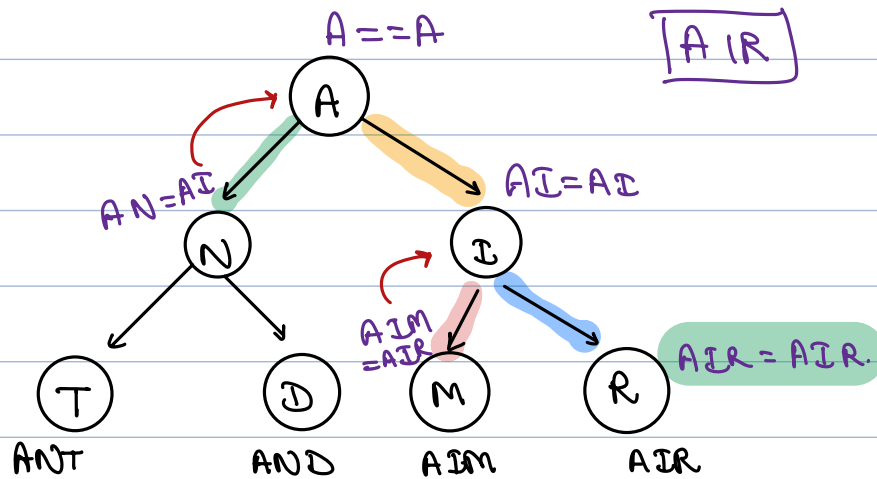
Backtracking:

Backtracking is like trying different paths. When you hit a dead end, you backtrack to the last choice and try a different route.

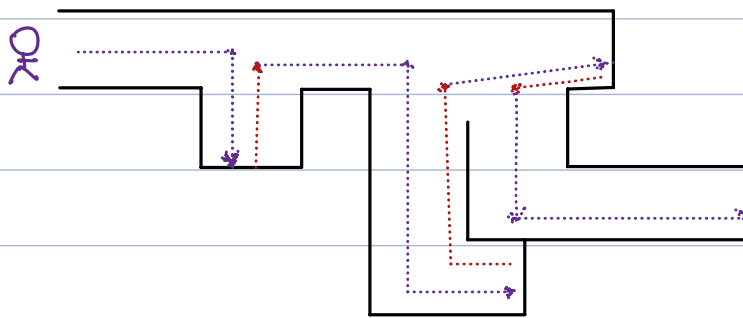


Find word "AIR"

Efficient way. : Backtracking
↳ Backtracking optimises the search, avoiding unnecessary exploration



Maze:



Idea: While generating all solutions, choose path but if we cannot go any further, backtrack and take another path.

Subsets

Subarray: Continuous part of array.

Subset: Any possible combination of the original array is a subset.

Subset:

↳ Set can be obtained by deleting 0 or more elements from the array.

↳ Empty set is a valid set

↳ Order does not matter here.

ex: {3, 2, 1, 9, 6, 8}

Subsets:

{1, 6, 8} $\xrightarrow{\text{Subset}}$ {*, *, 1, *, 6, 8} ✓

{9, 6, 3} $\xrightarrow{\text{Subset}}$ {3, *, *, 9, 6, *} ✓

{2, 9, 8} $\xrightarrow{\text{Subset}}$ {*, 2, *, 9, *, 8} ✓

Every subarray $\rightarrow$ subset
vice versa not valid

Subsequence

obtained by deleting elements from an array without changing order.

eg:

ar: [1, 2, 3, 4, 5]

	Subarray	Subset	Subsequence
{1, 2, 3, 4}	✓	✓	✓
{2, 3, 5}	X	✓	✓
{4, 1, 2}	X	✓	X
{2, 3}	✓	✓	✓
{1, 3, 5}	X	✓	✓

Note:

1. A subset may or may not be a subsequence

2. A subsequence is always a subset

3. A subarray is also a subsequence

Q → Given an array with distinct integers.

Print all subsets using recursion.

→ order does not matter (all possible combinations)

Eg. $arr = \{1, 2, 3\}$

Subsets: $[1], [1, 2], [1, 2, 3],$

$[2], [2, 3], [3], [1, 3], []$

$arr = \{6, 10\}$

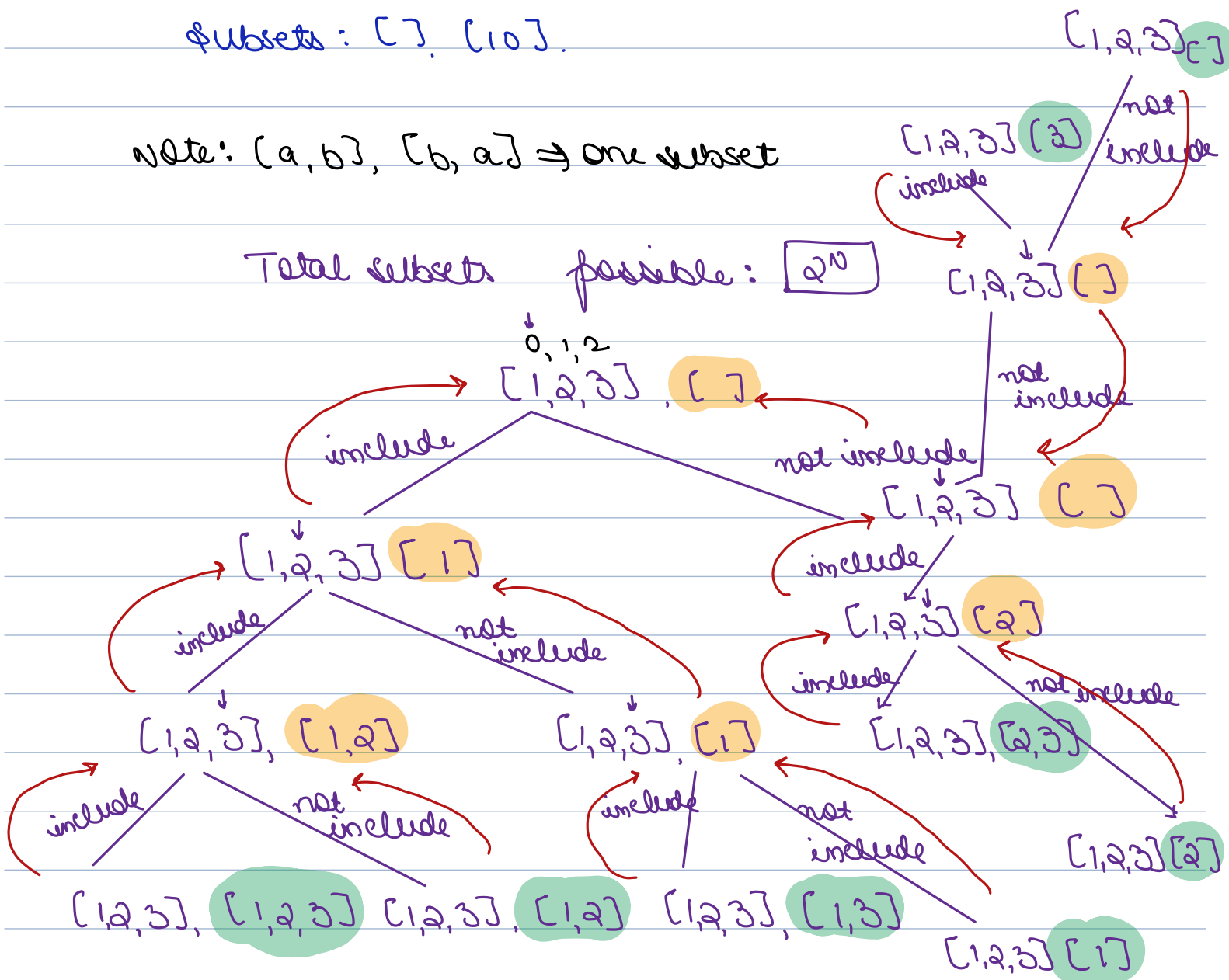
Subsets: $[6], [6, 10], [10], []$

$arr = \{10\}$

Subsets: $[], [10]$

Note: $[a, b], [b, a] \Rightarrow$ one subset

Total subsets possible: 2^n



Idea:

→ 2D list

list < ans >

TC: $O(2^N)$

SC: $O(N)$

void subsets(list A, list <> curset, int idx) {

```
    if (idx == A.length) {  
        ans.add(curset);  
        return;  
    }
```

// for every ele, we have 2 choices
// choice 1 → keep it in curset.

```
    curset.add(A[idx]);  
    subsets(A, curset, idx+1);
```

// choice 2 → Don't keep in curset
 curset.remove(A[idx]);
 subsets(A, curset, idx+1);

```
}
```

[[]]

[2,6,9], [], 0

Dry Run.

idx=0

```
if (idx == A.length) {  
    ans.add(curset);  
    return;  
}
```

```
curset.add(A[idx]);  
subset(A, curset, idx+1);  
curset.remove(A[idx]);  
subset(A, curset, idx+1);
```

idx=1

```
if (idx == A.length) {  
    ans.add(curset);  
    return;  
}
```

curset [2, 1]

```
curset.add(A[idx]);
```

- subset(A, curset, idx+1);
- curset.remove(A[idx]);
- subset(A, curset, idx+1);

idx=2

```
if (idx == A.length) {  
    ans.add(curset);  
    return;  
}
```

curset [2, 6, 1]

```
curset.add(A[idx]);
```

- subset(A, curset, idx+1);
- curset.remove(A[idx]);
- subset(A, curset, idx+1);

idx=2

```
if (idx == A.length) {  
    ans.add(curset);  
    return;  
}
```

- curset.add(A[idx]);
- subset(A, curset, idx+1)
- curset.remove(A[idx])
- subset(A, curset, idx+1)

curset [2, 9]



`idx = 3`

```
• if (idx == A.length) {  
  ans.add(curset);  
  return;  
}
```

```
curset.add(A[idx]);  
subset(A, curset, idx+1);  
curset.remove(A[idx]);  
subset(A, curset, idx+1);
```

ans: `[[2, 6, 9], [2, 8]]`

→ TODO complete
dry run

{ Backtracking → changes you have done while
going down in recursion. undo that changes

a, b, c → length 3

abc

acb

bac

bca

cab

cba

→ `6`

`3! ⇒ 6`

`N!`

ab

ba

ba

2! ⇒ `2`

Permutations:

Given a character array with distinct elements.

Print all permutations of it without modifying it.

inp: [a, b, c]

Permutations,

abc

acb

bac

bec

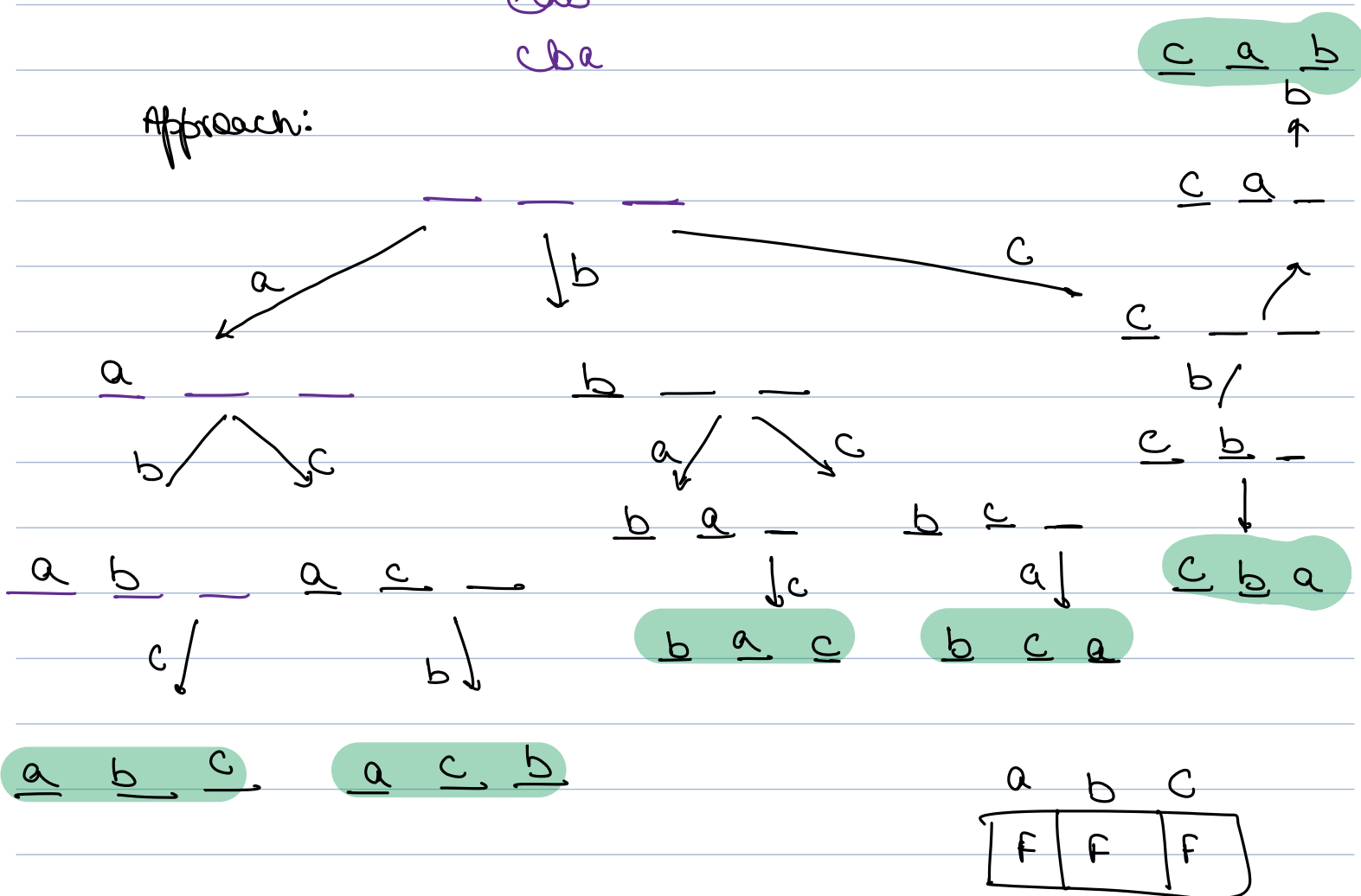
cab

cba

$N=3 \rightarrow 6$ permutations

$[3!]$

Approach:



code

```
void permutations(char[] ar, ans[N], idr, boolean visited[]){
```

```
    if(idr == ar.length){
```

```
        print(ans);
```

```
        return;
```

```
    }
```

0	1	2
F	F	F

```
    for(int i=0; i<N; i++){ // all possibilities
```

```
        if(visited[i] == false){
```

```
            visited[i] = true;
```

```
            ans[i] = ar[i];
```

```
            permutations(ar, ans, idr+1, visited[]);
```

```
            visited[i] = false; // undo changes.
```

↳ backtracking

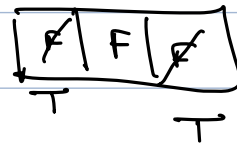
```
        }
```

```
    }
```

Tc: $O(2^N)$

Sc: $O(N)$

$idx = 0,$



```
if (idx == arr.length) {  
    print(ans);  
    return;  
}
```

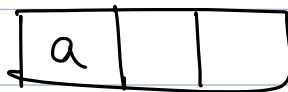
```
for (int i = 0; i < n; i++) { // all possibilities  
    if (visited[i] == false) {  
        visited[i] = true;  
        ans[idx] = arr[i];  
        permutations(arr, ans, idx + 1, visited);  
        visited[i] = false; // undo changes.  
    }  
}
```



```
if (idx == arr.length) {  
    print(ans);  
    return;  
}
```

$i = 1$

```
for (int i = 0; i < n; i++) { // all possibilities  
    if (visited[i] == false) {  
        visited[i] = true;  
        ans[idx] = arr[i];  
        permutations(arr, ans, idx + 1, visited);  
        visited[i] = false; // undo changes.  
    }  
}
```



$i = 2$



```
if (idx == arr.length) {  
    print(ans);  
    return;  
}
```

$i = 3$

```
for (int i = 0; i < n; i++) { // all possibilities  
    if (visited[i] == false) {  
        visited[i] = true;  
        ans[idx] = arr[i];  
        permutations(arr, ans, idx + 1, visited);  
        visited[i] = false; // undo changes.  
    }  
}
```



$i=3$

```
if (idx == arr.length) {  
    print(ans);  
    return;  
}
```

a, b, c

```
for (int i=0; i<n; i++) { // all possibilities  
    if (visited[i] == false) {  
        visited[i] = true;  
        ans[i] = arr[i];  
        permutations(arr, ans, idx+1, visited);  
        visited[i] = false; // undo changes.  
    }  
}
```

a	b	c
F	F	F

$i=0$

T	F	F
---	---	---

a b,

$i=1$

T	F	F
---	---	---

T	F	T
---	--------------	---

$i=2$

T	T	F
---	---	---

T

T	T	T
---	---	---

$i=3$

acb

abc

N-Queens

Ques. Given $N \times N$ matrix, print all valid placement of N Queens such that no queen can kill other queen.

Note: if 2 queens belong to the same row/column/diagonal they will kill.

$N=4$

	0	1	2	3
0	Q			
1			Q	
2		Q		
3				

	0	1	2	3
0		Q		
1				Q
2	Q			
3			Q	

	0	1	2	3
0			Q	
1	Q			
2				Q
3		Q		

	X	X		
0				
1				
2				
3				

- - - TODO

	0	1	2	3
0				
1				
2				
3				

	0	1	2	3
0				
1				
2				
3				

	0	1	2	3
0				
1				
2				
3				

	0	1	2	3
0				
1				
2				
3				

	0	1	2	3
0				
1				
2				
3				

	0	1	2	3
0				
1				
2				
3				

	0	1	2	3
0				
1				
2				
3				

	0	1	2	3
0				
1				
2				
3				

	0	1	2	3
0				
1				
2				
3				

	0	1	2	3
0				
1				
2				
3				

	0	1	2	3
0				
1				
2				
3				

	0	1	2	3
0				
1				
2				
3				

	0	1	2	3
0				
1				
2				
3				